



中国科学技术大学
University of Science and Technology of China

计算系统概论A

Introduction to Computing Systems
(CS1002A.03)

Chapter 8 Data Structures

陈俊仕

cjuns@ustc.edu.cn

2025 Fall

计算机科学与技术学院

School of Computer Science and Technology

Exercise 4

7.24 We want the following program fragment to shift R3 to the left by four bits, but it has an error in it. Identify the error and explain how to fix it.

```
        .ORIG  x3000
        AND    R2, R2, #0
        ADD    R2, R2, #4
LOOP    BRz    DONE
        ADD    R2, R2, #-1
        ADD    R3, R3, R3
        BR     LOOP
DONE    HALT
        .END
```

★7.37 During the processing of an LC-3 program by the data path we have been using in class, the computer stops due to a breakpoint set at x3000. The contents of certain registers and memory locations at that time are as follows:

R2 through R7: x0000
M[x3000]: x1263
M[x3003]: x0000

The LC-3 is restarted and executes exactly four instructions. To accomplish this, a number of clock cycles are required. In 15 of those clock cycles, the bus must be utilized. The following table lists those 15 clock cycles in sequential order, along with the values that are gated onto the LC-3 bus in each.

Fill in the missing entries above.

What are the four instructions that were executed?

What are the contents of R0 and R1 after the four instructions execute?

	BUS
1st:	x3000
2nd:	x1263
3rd:	x009A
4th:	x3001
5th:	xA000
6th:	
7th:	
8th:	
9th:	
10th:	
11th:	
12th:	
13th:	x3003
14th:	x1263
15th:	x009D

Exercise 5

6.18 The LC-3 has no Divide instruction. A programmer needing to divide two numbers would have to write a routine to handle it. Show the systematic decomposition of the process of dividing two positive integers. Write an LC-3 machine language program starting at location x3000 that divides the number in memory location x4000 by the number in memory location x4001 and stores the quotient at x5000 and the remainder at x5001.

1

Review

2

Subroutines

3

Control Instructions for Subroutines

4

Memory Model for Program Execution

5

The Stack

6

Implementing Functions in C

1 Review

2 Subroutines

3 Control Instructions for Subroutines

4 Memory Model for Program Execution

5 The Stack

6 Implementing Functions in C

Abstract Data Types: Data Structures

■ Up to now, we have processed a single value

- an integer
- a floating point number, or
- an ASCII character

■ The information in the real world is far more complex than simple, single numbers. We call these complex items of information **abstract data types**, or more colloquially **data structures**, far more complex than simple, single numbers. E.g.

- a company's organization chart
- a list of items arranged in alphabetical order

■ In this chapter, we will study three abstract data types:

- stacks
- queues
- and character strings

Abstract Data Types: Data Structures

- We will write programs to solve problems that require expressing information according to its structure.
- Before we get to **stacks, queues, and character strings**, however, we introduce a new concept that will prove very useful in manipulating data structures: **subroutines**, or what is also called **functions**.

1

Review

2

Subroutines

3

Control Instructions for Subroutines

4

Memory Model for Program Execution

5

The Stack

6

Implementing Functions in C

Subroutines

■ A subroutine is a program fragment that...

- Resides in **user space** (*i.e.*, not in OS)
- Performs a **well-defined task**
- Is invoked (called) **multiple times** by a user program
- **Returns control** to the calling program when finished

■ Virtues

- **Reuse code** without re-typing it (and debugging it!)
- Divide task into parts (or among multiple programmers)
- Use vendor-supplied **library** of useful routines that one software engineer writes a program that requires such fragments and another software engineer writes the fragments.
 - math library
 - square root, sine, and arctangent, etc.

■ In C language, called **function**; In other languages, called **procedures, subroutines, methods ...**

A simple illustration of a part of a program

```

01; Service Routine for Keyboard Input
02      .ORIG      x04A0          ;System call starting address
03
04;START  ST          R7,SaveR7      ;Save the linkage back to the
05;                               ;program?
06      ST          R1,SaveR1        ;Save the values in the registers
07      ST          R2,SaveR2        ;that are used so that they can
08      ST          R3,SaveR3        ;be restored before RET
09
10;Output Newline on CRT
11      LD          R2,Newline
12 L1    | LDI          R3,DSR          ;Check DDR—is it free?
13      | BRzp        L1              ;Loop until monitor is ready
14      | STI          R2,DDR          ;Move cursor to new clean line
15;
16;Output "Input a character"
17      LEA          R1,Prompt        ;Prompt is starting address
18      ;of prompt string
19 Loop  LDR          R0,R1,#0        ;Get next prompt character
20      BRzp        Input            ;Check for end of prompt string
21 L2    | LDI          R3,DSR
22      | BRzp        L2
23      | STI          R0,DDR          ;Write next character of prompt
24      ;string
25      ADD          R1,R1,#1         ;Increment prompt point
26      BRnzp       Loop
  
```

Label	LDI	R3,DSR
	BRzp	Label
	STI	Reg,DDR

A simple illustration of a part of a program

27;Input a character from KB

28 Input LDI R3,KBSR ;Has a character been typed?

29 BRzp Input

30 LDI R0,KBDR ;Load it into R0

31

32;Echo the character on CRT

33 L3 | LDI R3,DSR

34 | BRzp L3

35 | STI R0,DDR ;Echo input character to the

36 | ;monitor

37;Output Newline on CRT

38 L4 | LDI R3,DSR ;Check CRTDR—is it free?

39 | BRzp L4

40 | STI R2,DDR ;Move cursor to new clean line

41

42;Restore

43 LD R1,SaveR1 ;Service routine done, restore

44 LD R2,SaveR2 ;original values in registers.

45 LD R3,SaveR3 ;

46; LD R7,SaveR7 ;Restore linkage back

47; ;prior to RET?

48 RET ;Return to calling program

Label

LDI R3,DSR

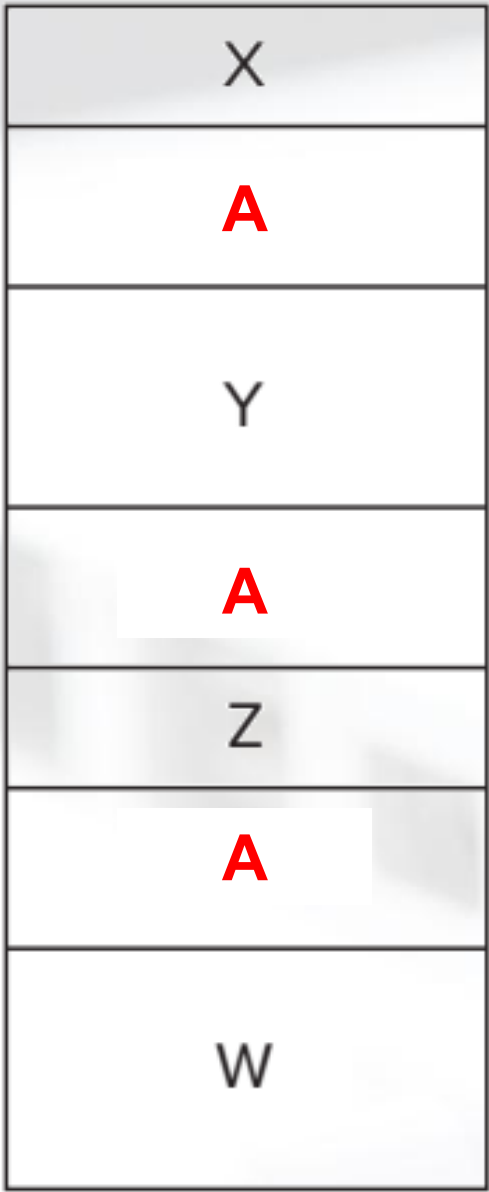
BRzp Label

STI Reg,DDR

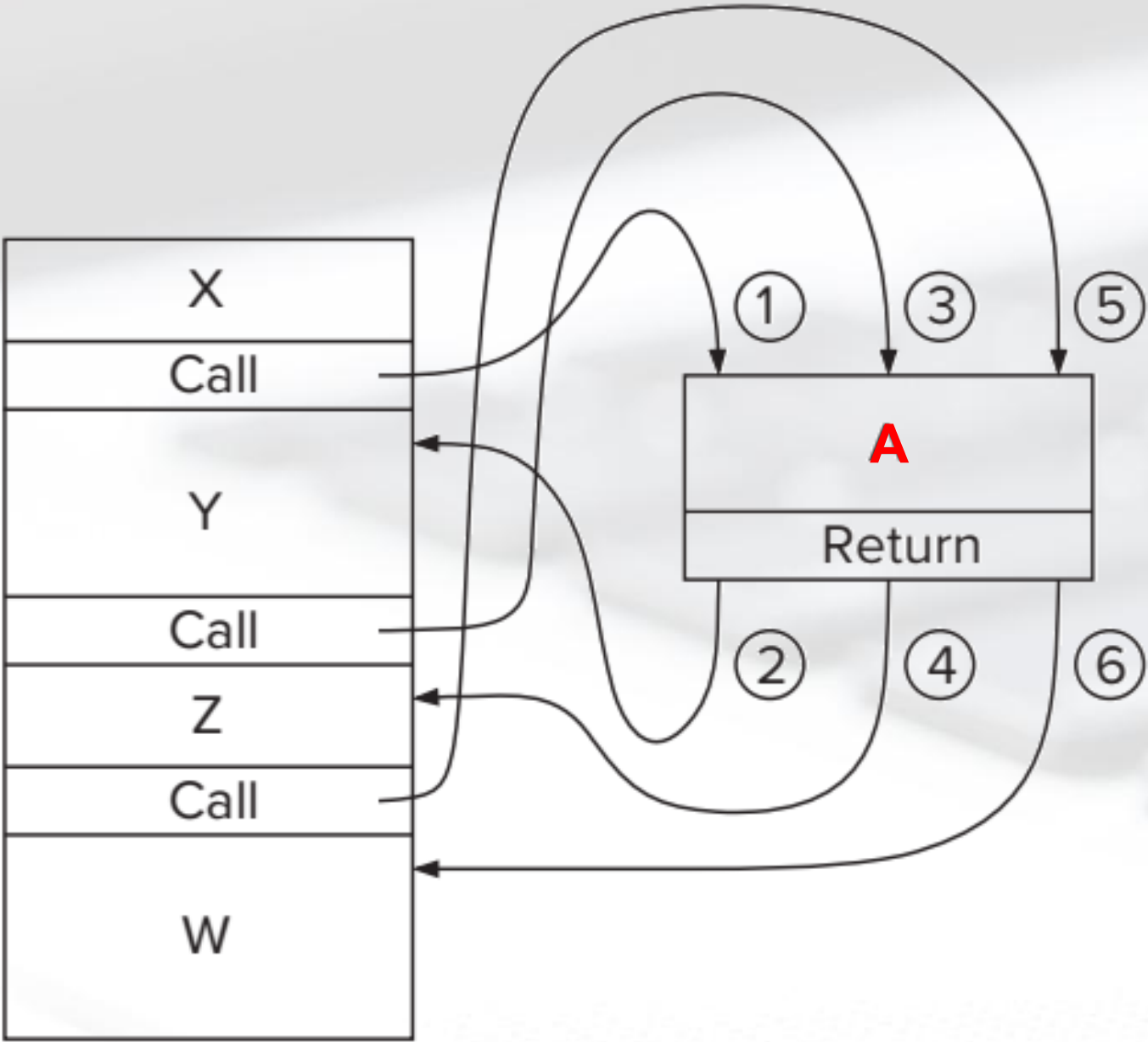
A simple illustration of a part of a program

```
49;Memory for registers saved
50;    SaveR7    .FILL x0000
51    SaveR1    .FILL x0000
52    SaveR2    .FILL x0000
53    SaveR3    .FILL x0000
54
55    DSR        .FILL xF3FC
56    DDR        .FILL xF3FF
57    KBSR       .FILL xF400
58    KBDR       .FILL xF401
59 ;
59    Newline    .FILL x000A    ;ASCII code for newline
60    Prompt     .STRINGZ "Input a character>"
61                .END
```

The Call/Return Mechanism



(a) Without subroutines



(b) With subroutines

1 Review

2 Subroutines

3 **Control Instructions for Subroutines**

4 Memory Model for Program Execution

5 The Stack

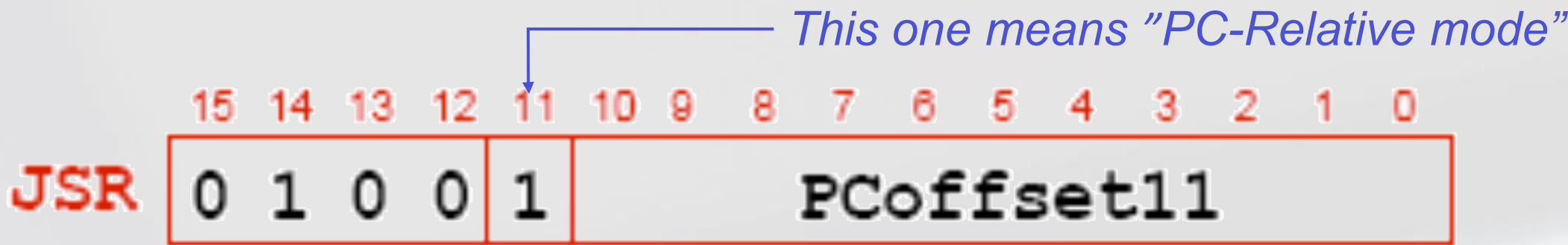
6 Implementing Functions in C

Control Instructions for Subroutines



	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BR	0	0	0	0	n	z	p	PCoffset9								
JSR	0	1	0	0	1	PCoffset11										
JSRR	0	1	0	0	0	0	0	BaseR		0	0	0	0	0	0	0
RTI	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
JMP	1	1	0	0	0	0	0	BaseR		0	0	0	0	0	0	0
RET	1	1	0	0	0	0	0	1	1	1	0	0	0	0	0	0
TRAP	1	1	1	1	0	0	0	0	TrapVector8							

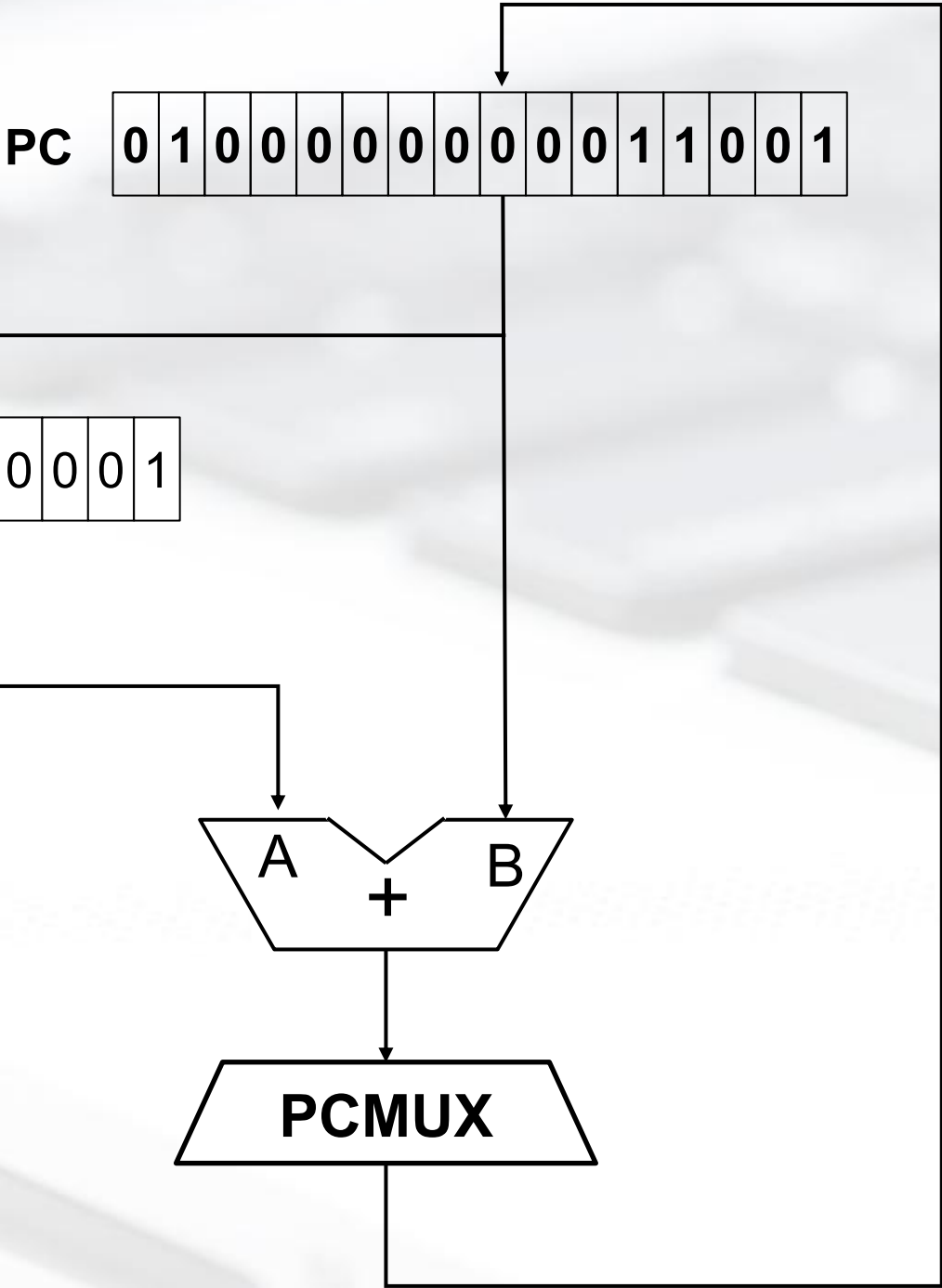
JSR (PC-Relative)



Register File

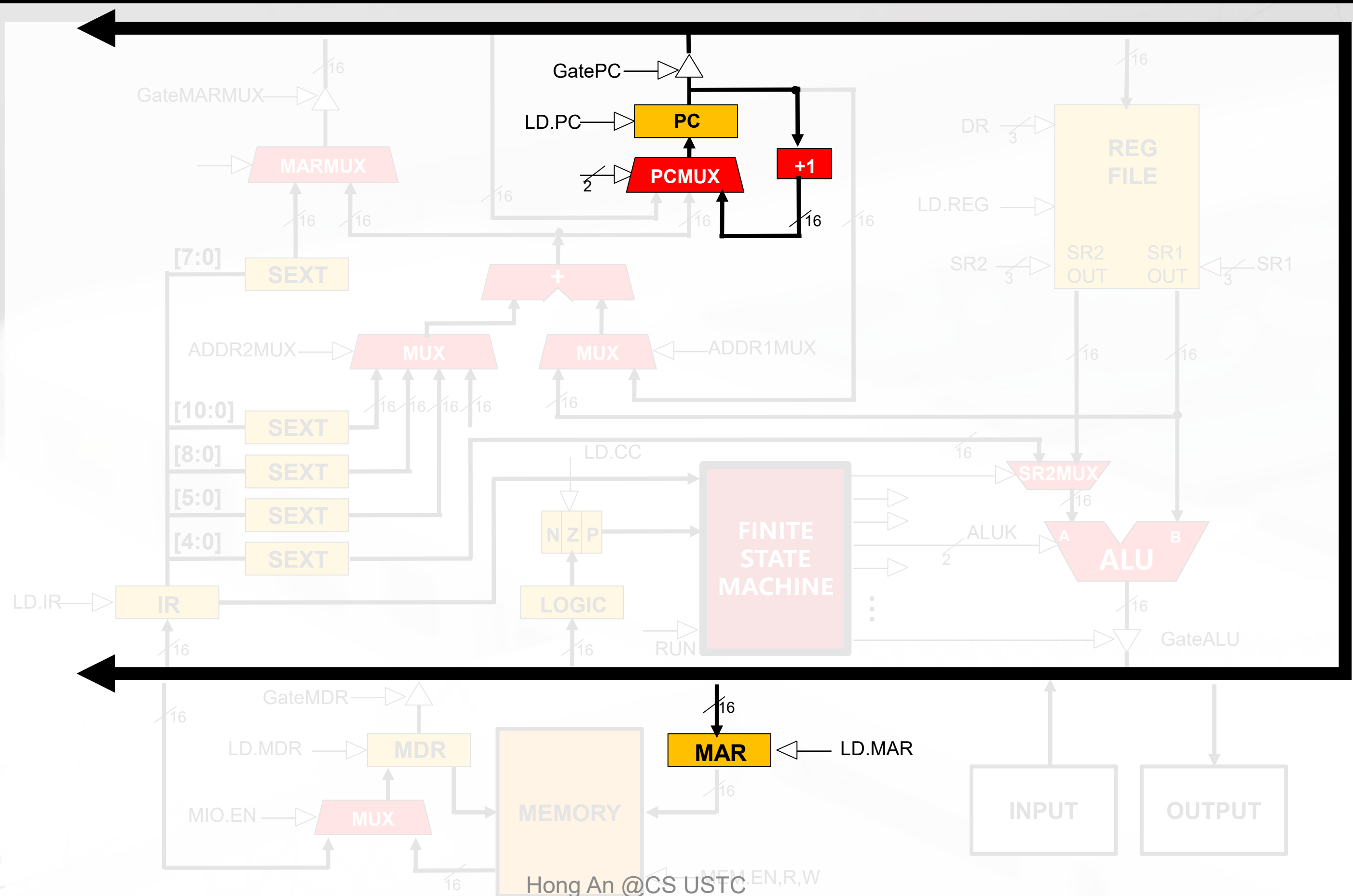
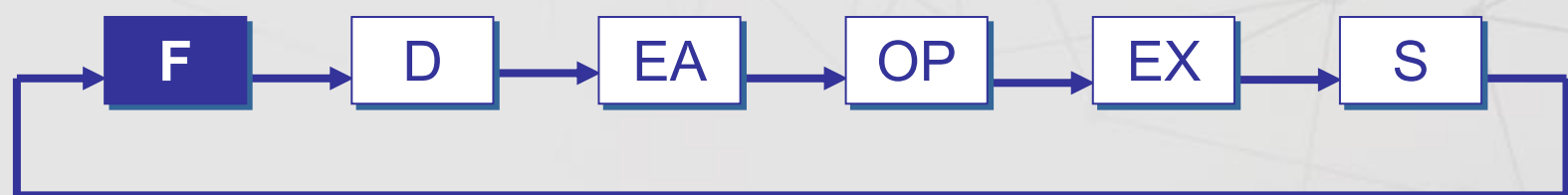
R0	
R1	
R2	
R3	
R4	
R5	
R6	
R7	0 1 0 0 0 0 0 0 0 0 0 0 1 1 0 0 1

Note: This is PC of next instruction

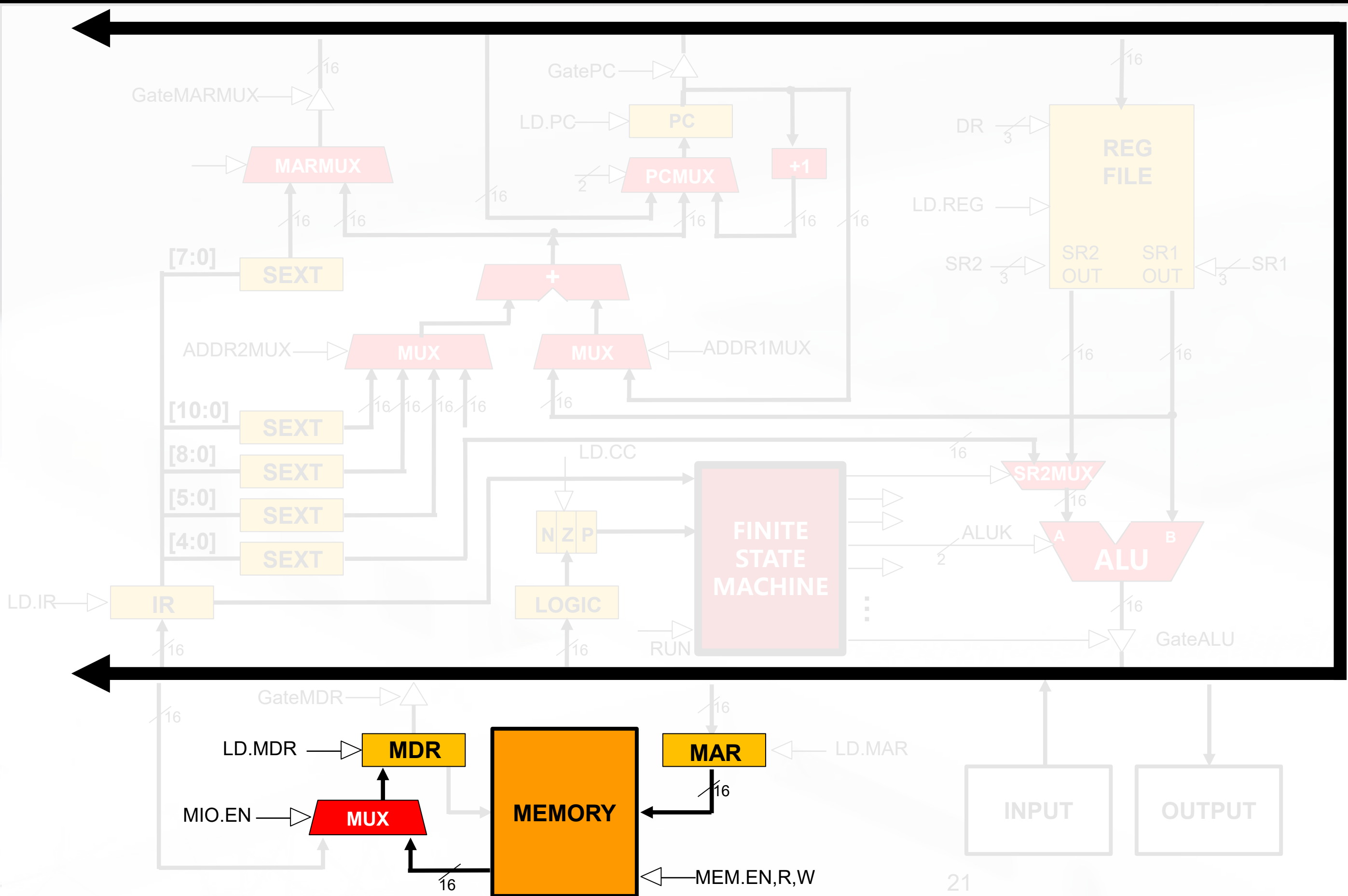
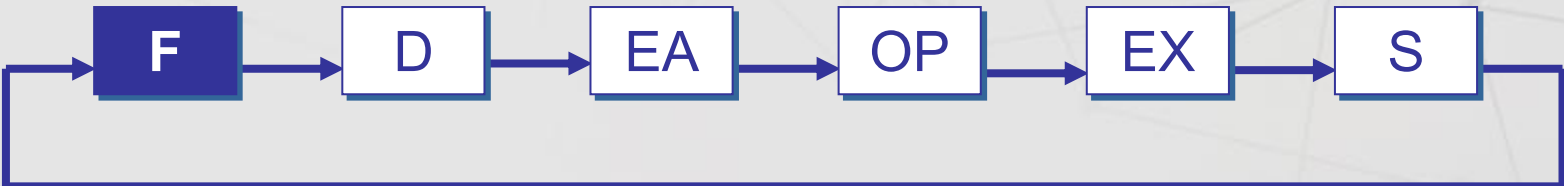


Just like JMP (but PC is saved in R7)
Why not just use TRAP?

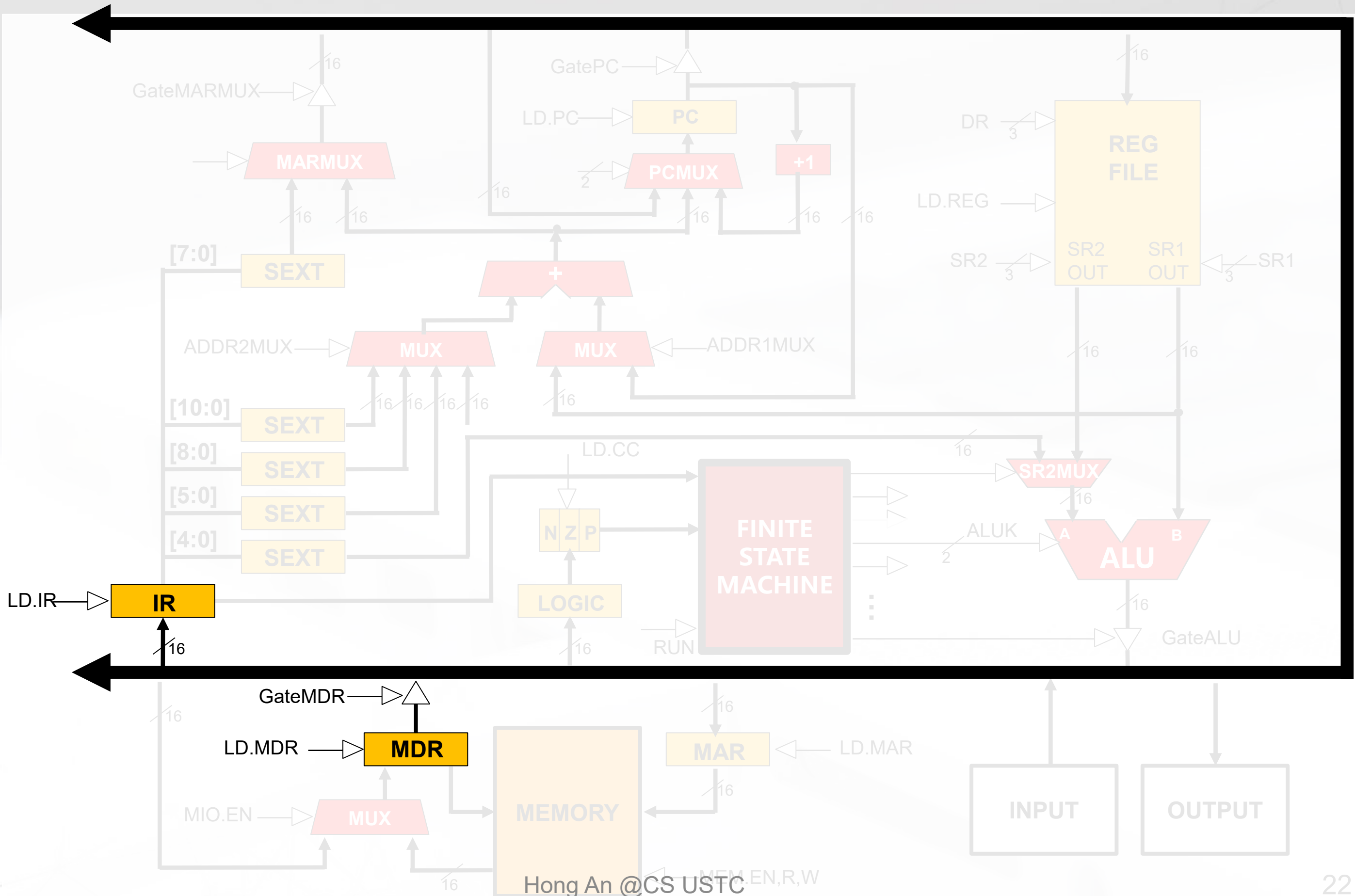
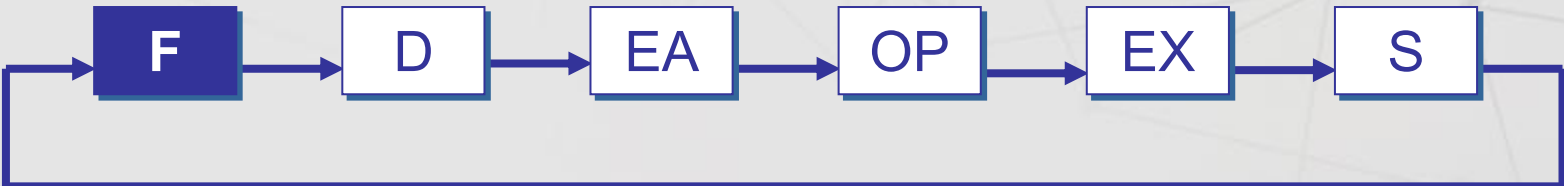
JSR (PC-Relative)



JSR (PC-Relative)



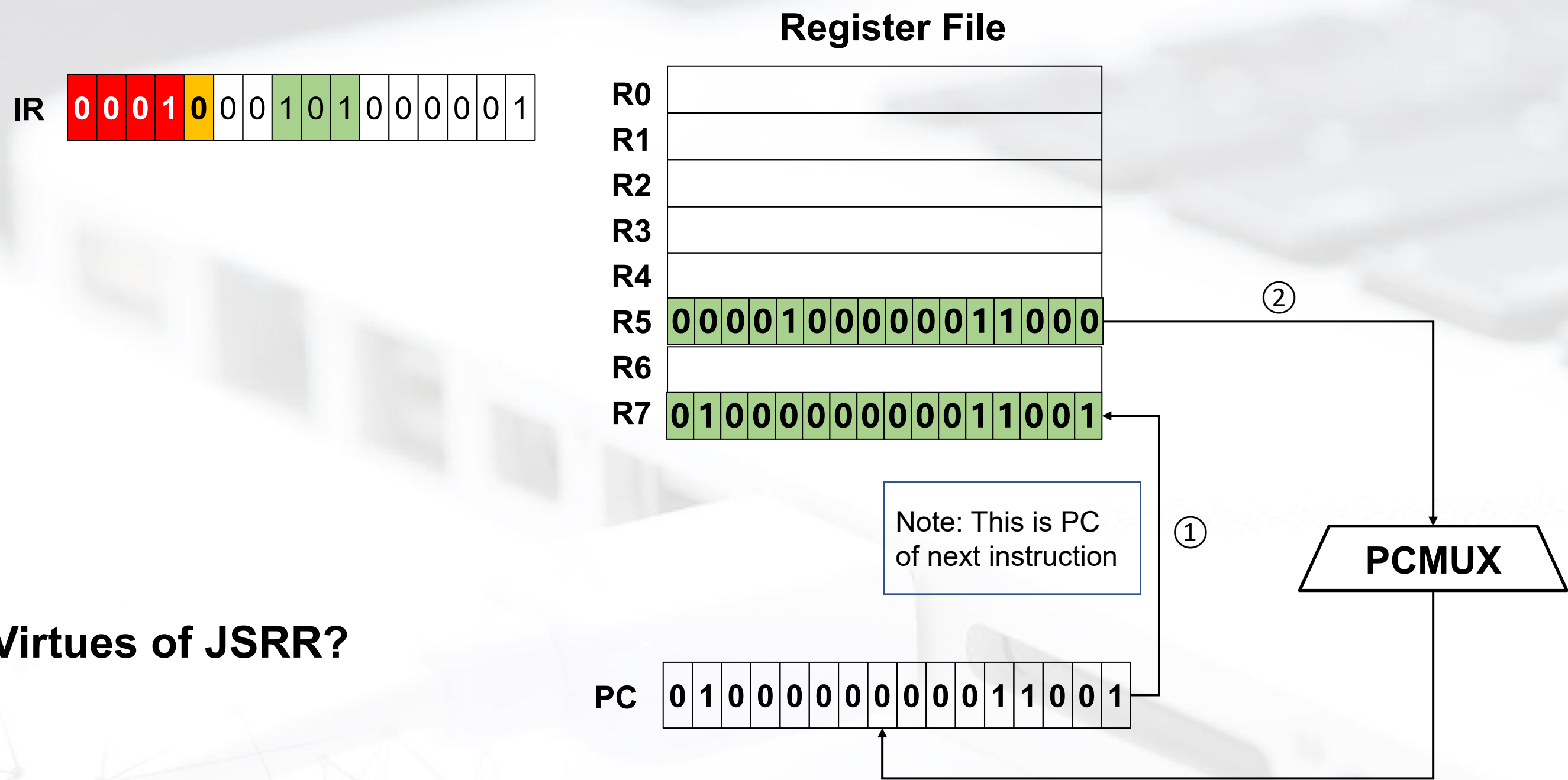
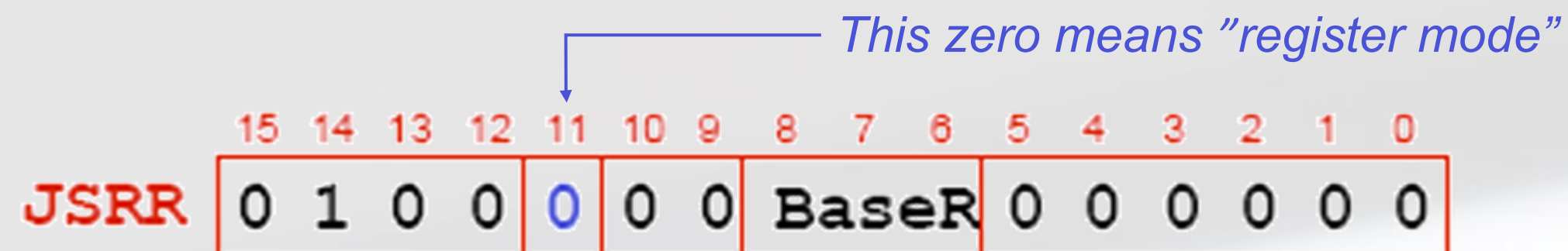
JSR (PC-Relative)



The logo of the University of Science and Technology of China (USTC) is a circular emblem. It features a blue outer ring with the university's name in Chinese characters '中国科学技术大学' at the top and 'University of Science and Technology of China' at the bottom. The center of the logo is a stylized blue design containing the year '1958' and a graphic of an open book with a sail rising from it, symbolizing knowledge and progress.

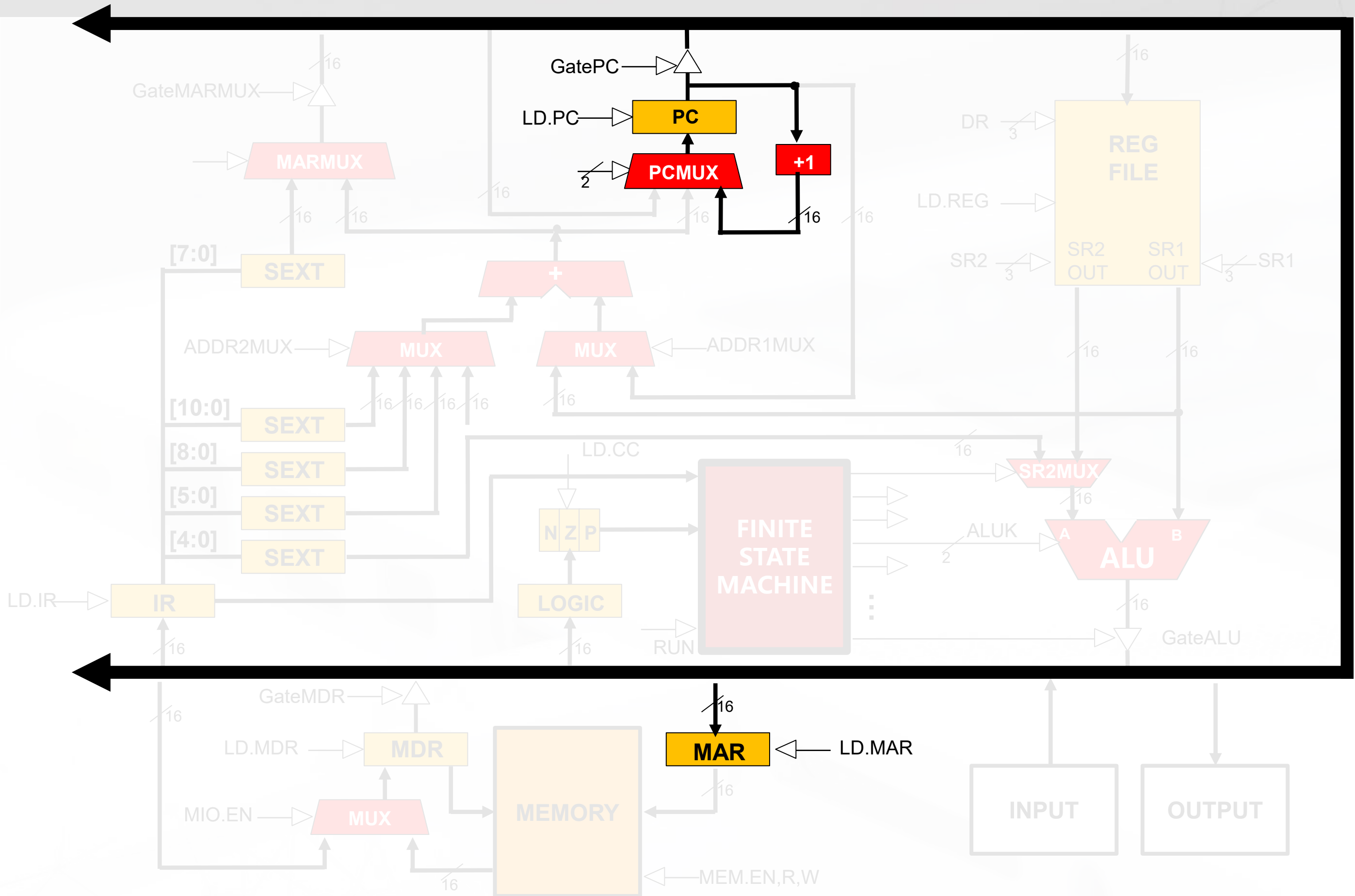
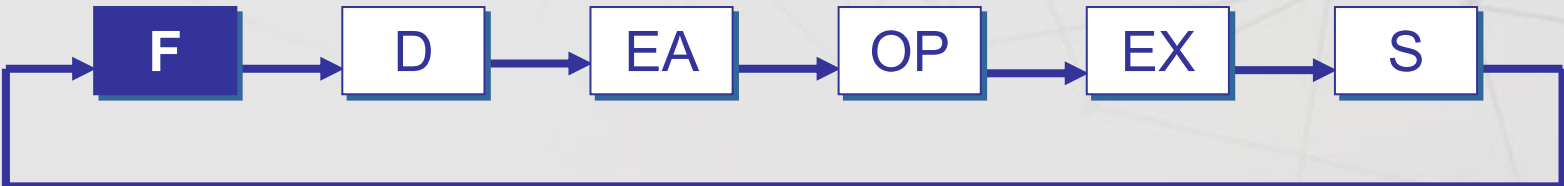
The logo of the University of Science and Technology of China (USTC) is a circular emblem. It features a blue outer ring with the university's name in Chinese characters '中国科学技术大学' at the top and 'University of Science and Technology of China' at the bottom. The center of the logo is a stylized blue design containing the year '1958' and a graphic of an open book with a sail-like shape rising from it.

JSRR (Register)

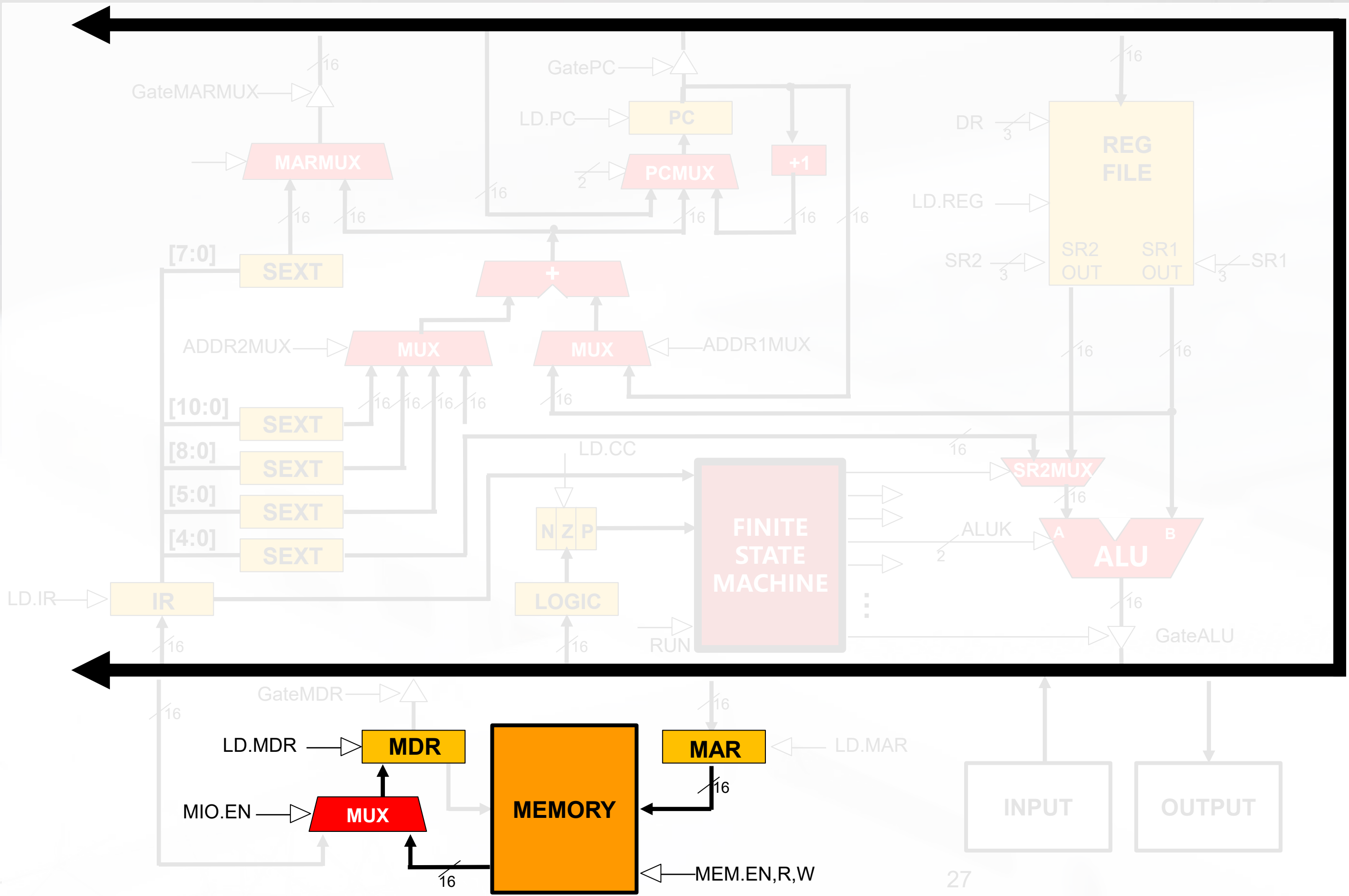
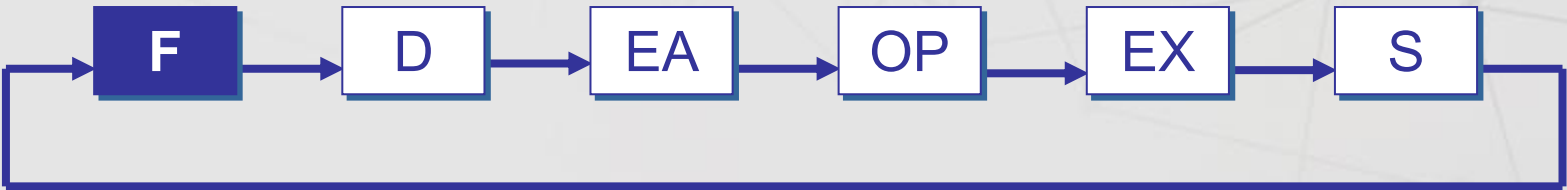


Virtues of JSRR?

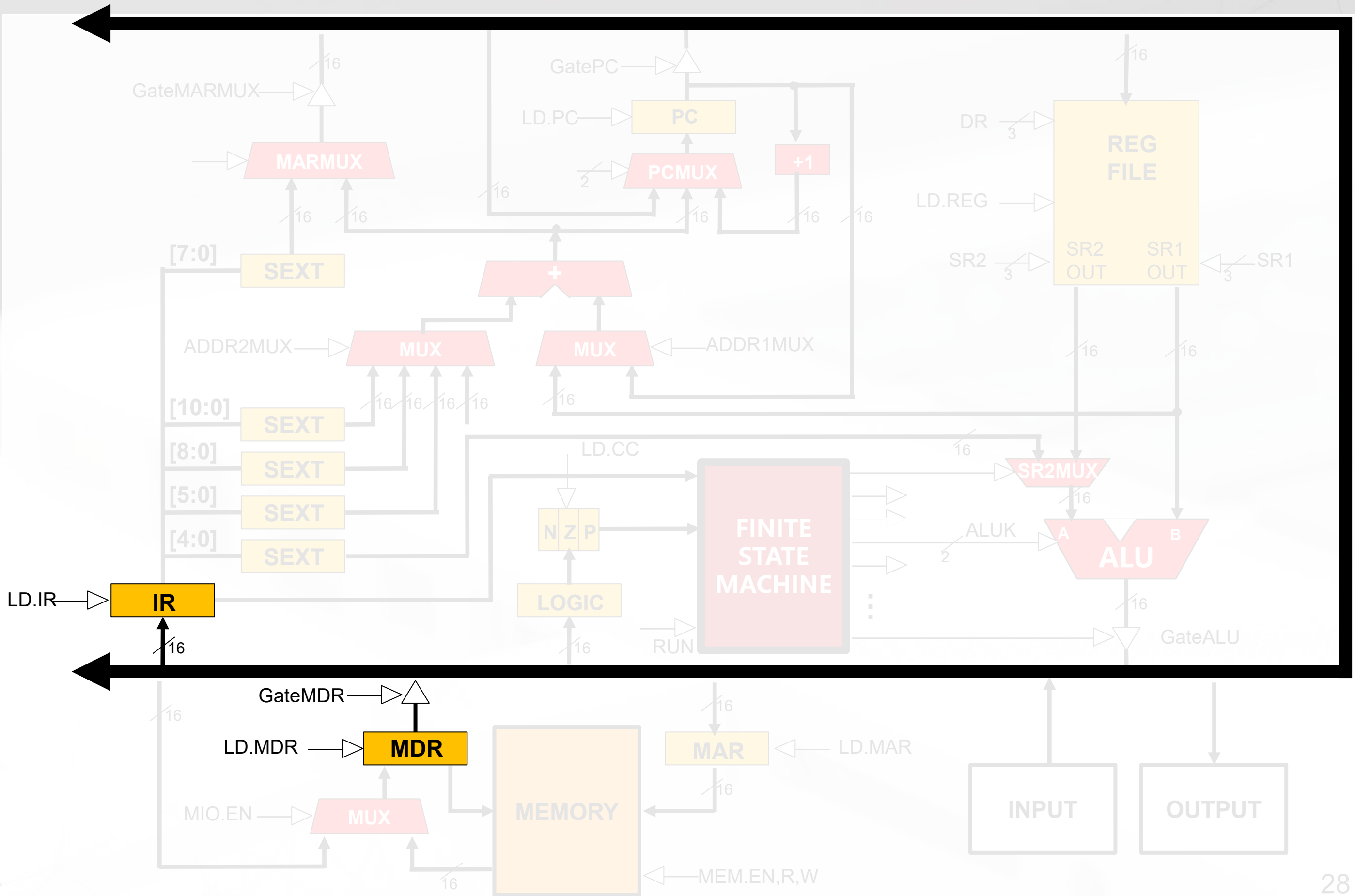
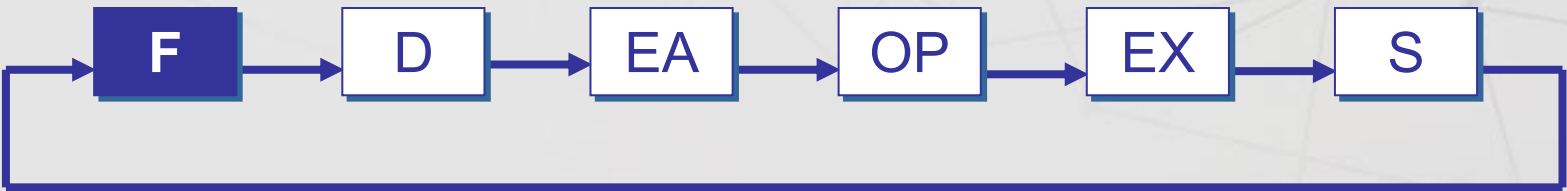
JSRR (Register)



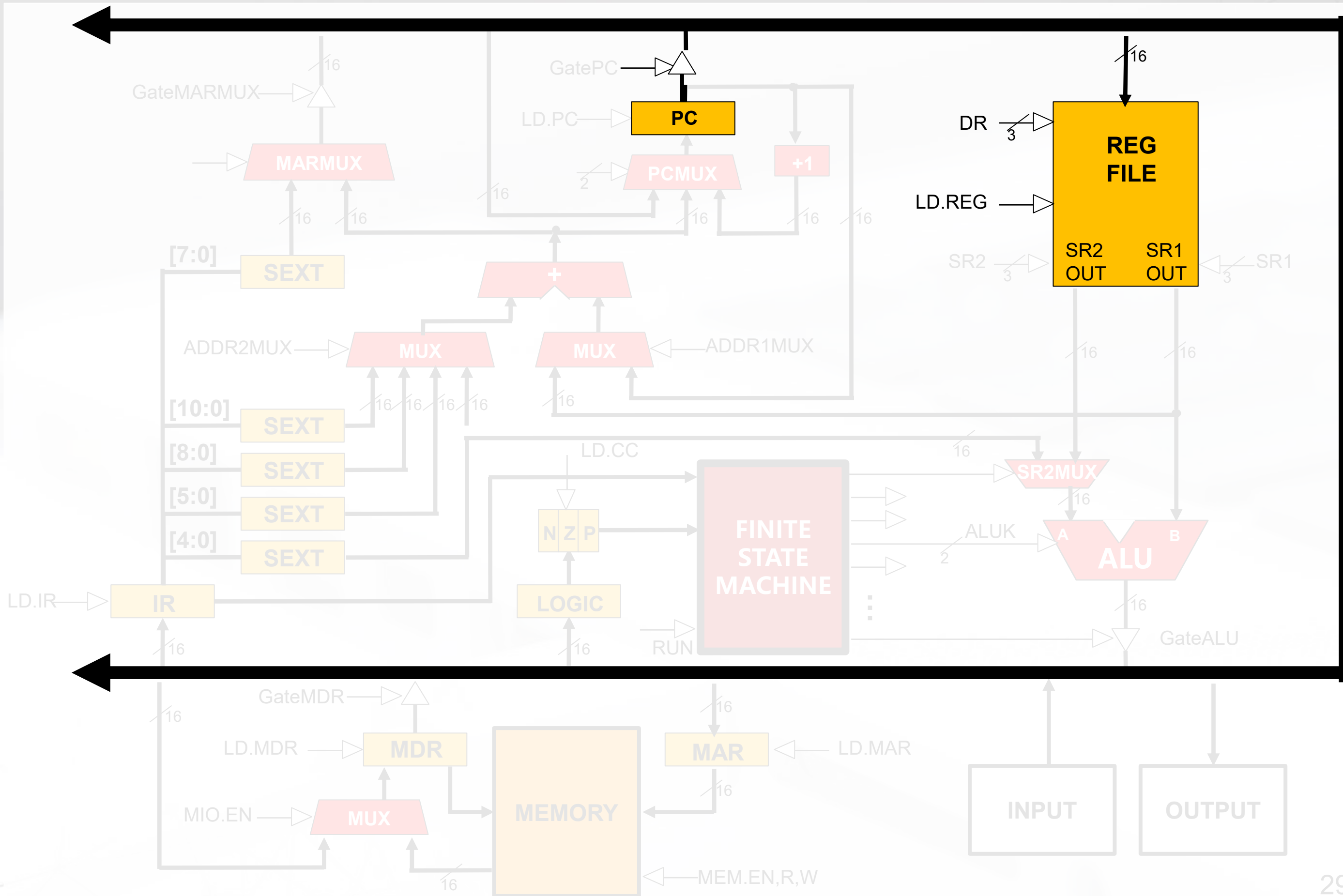
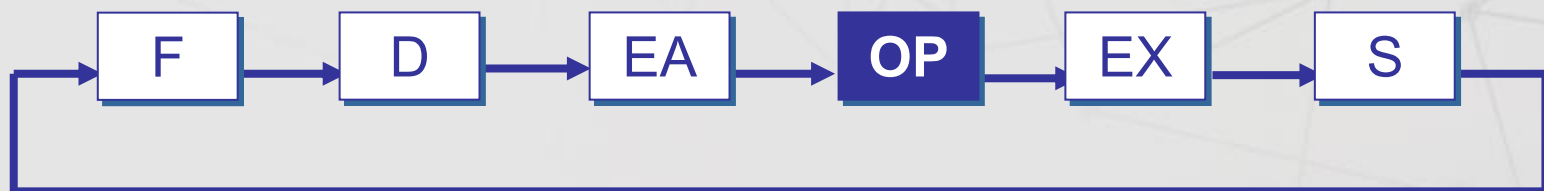
JSRR (Register)



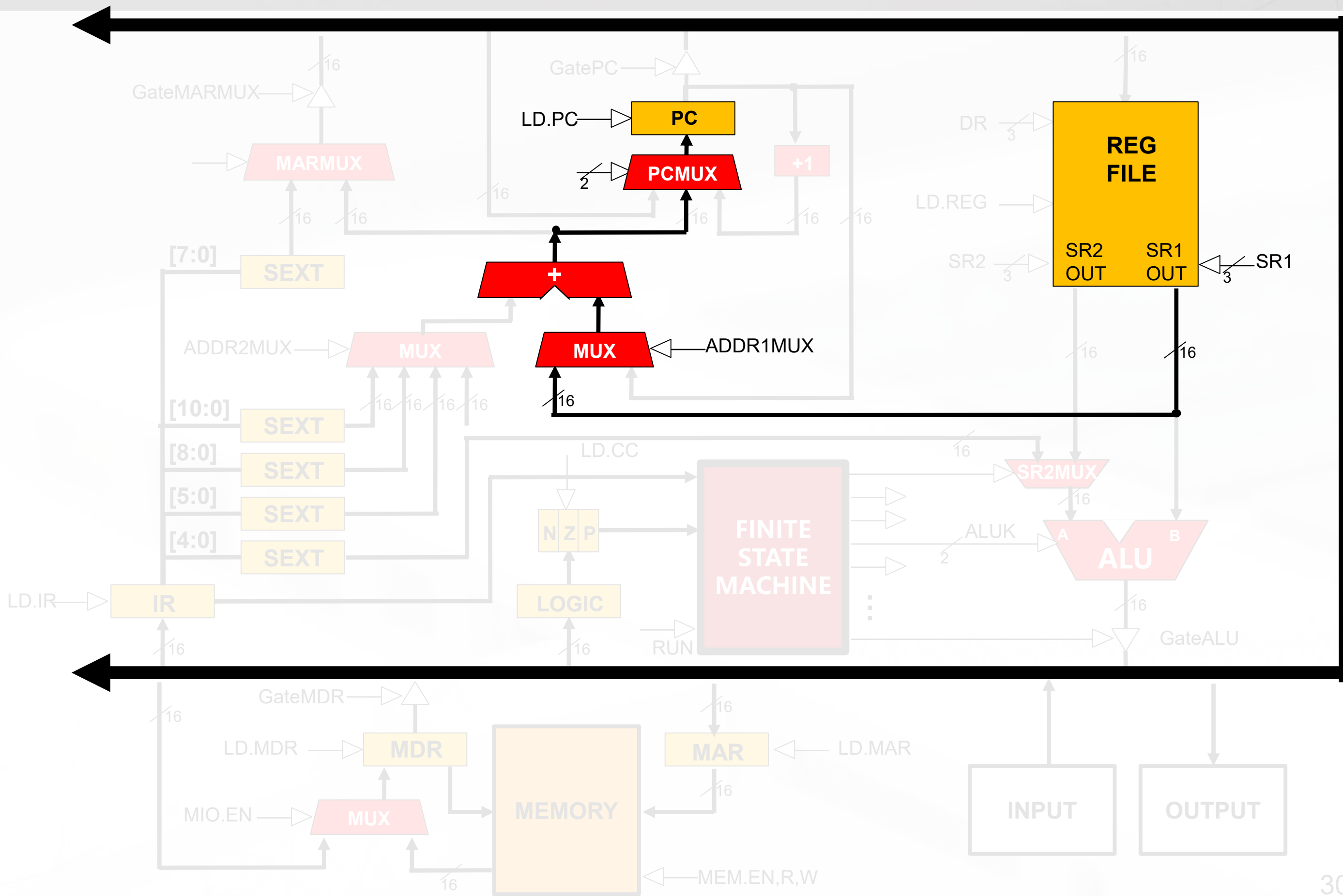
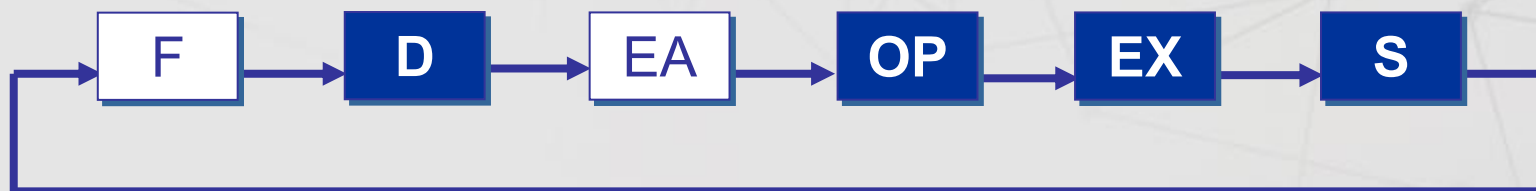
JSRR (Register)



JSRR (Register)



JSRR (Register)



RET instruction

■ RET – return instruction

● How to return

— Place address in R7 in PC, Return the execution to the last calling point.

● $PC \leftarrow (R7)$

RET
(JMP R7)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	1	0	0	0	0	0	0

Example: Negate the value in R0

```
TwosComp    NOT    R0,R0        ;flip bits
             ADD    R0,R0,#1     ;add one
             RET                      ;return to caller
```

To call from a program

```
;need to compute R4 = R1-R3
             ADD    R0,R3,#0      ;copy R3 to R0
             JSR    TwosComp      ;negate
             ADD    R4,R1,R0      ;add to R1
             ...
```

6.17 Shown below are the contents of registers before and after the LC-3 instruction at location x3210 is executed. Your job: Identify the instruction stored in x3210. *Note:* There is enough information below to uniquely specify the instruction at x3210.

	Before	After
R0:	xFF1D	xFF1D
R1:	x301C	x301C
R2:	x2F11	x2F11
R3:	x5321	x5321
R4:	x331F	x331F
R5:	x1F22	x1F22
R6:	x01FF	x01FF
R7:	x341F	x3211
PC:	x3210	x3220
N:	0	0
Z:	1	1
P:	0	0

Using Subroutines

■ Programmer must know

- **Address:** or at least a label that will be bound to its address
- **Function:** what it does
 - NOTE: The programmer does not need to know *how* the subroutine works, but what changes are visible in the machine's state after the routine has run
- **Arguments:** what they are and where they are placed
- **Return values:** what they are and where they are placed

Passing Information To Subroutines

■ Argument(s)

- Value **passed in** to a subroutine is called an argument
- This is a value needed by the subroutine to do its job
- Examples
 - TwosComp: R0 is number to be negated
 - OUT: R0 is character to be printed
 - PUTS: R0 is *address* of string to be printed

■ How?

- In registers (simple, fast, but limited number)
- In memory (many, but awkward, expensive)
- Both

Getting Values From Subroutines

■ Return Values

- A value **passed out** of a subroutine is called a return value
- This is the value that you called the subroutine to compute
- **Examples**
 - TwosComp: negated value is returned in R0
 - GETC: character read from the keyboard is returned in R0

■ How?

- Registers, memory, or both
- Single return value in register most common

A problem when dealing with subroutines

- We have known that every time an instruction loads a value into a register, the value that was previously in that register is lost. Thus, we need to save the value in a register
 - if that value will be destroyed by some subsequent instruction, and
 - if we will need it after that subsequent instruction.
- Caution Using JSR, JSRR, and TRAP
 - You MUST save R7 if you call any other subroutine using JSR, JSRR or TRAP

Caution Using TRAPs (*“caller-save” in User code*)

	LEA	R3,BLOCK	;Init. To first loc.
	LD	R6,ASCII	;Char->digit template
	LD	R7,COUNT	;Init. to 10
AGAIN	TRAP	x23	;Get char
	ADD	R0,R0,R6	;Convert to number
	STR	R0,R3,#0	;Store number
	ADD	R3,R3,#1	;Incr pointer
	ADD	R7,R7,-1	;Decr counter
	BRp	AGAIN	;More?
	BRnzp	NEXT_TASK	
ASCII	.FILL	xFFD0	;Negative of x0023
COUNT	.FILL	#10	
BLOCK	.BLKW	#10	

Caution Using TRAPs (*“caller-save” in User code*)

```

AGAIN      LEA      R3,BLOCK      ;Init. To first loc.
           LD       R6,ASCII      ;Char->digit template
           LD       R7,COUNT      ;Init. to 10
           ST       R7,SaveR7
           TRAP     x23           ;Get char
           LD       R7,SaveR7
           ADD      R0,R0,R6      ;Convert to number
           STR      R0,R3,#0      ;Store number
           ADD      R3,R3,#1      ;Incr pointer
           ADD      R7,R7,-1      ;Decr counter
           BRp      AGAIN        ;More?
           BRnzp    NEXT_TASK

SaveR7    .BKLW 1

ASCII      .FILL    xFFD0        ;Negative of x0023
COUNT     .FILL    #10
BLOCK      .BLKW    #10

```

Saving and Restoring Registers

■ Called routine => *"callee-save"*

- Before start, save registers that will be altered
(except output regs)
- Before return, restore those same registers
(again, except output regs)
- Values are saved by storing them in memory

■ Calling routine => *"caller-save"*

- If register value needed later, save register destroyed by own instructions or by called routines (if known)
 - Save R7 before TRAP
- Or avoid using those registers altogether

■ *LC-3: By convention, callee-saved when possible*

- Other ISAs use a more efficient combination of caller- and callee-save

Saving and Restore Registers

■ Like service routines, must save and restore registers

- Who saves what is part of the calling convention

■ Generally, use “callee-save” strategy, except for return values

- Same as trap service routines
- Save anything that subroutine alters internally that shouldn't be visible when the subroutine returns
- Restore incoming arguments to original values (unless overwritten by return value)

■ Remember

- You MUST save R7 if you call any other subroutine or trap
- Otherwise, you won't be able to return!

Subroutine Template

```
01 SUB_NAME
02     ;Register Saving
03     ST R0, SUB_R0
04     ST R1, SUB_R1
05     ...
06     ST R6, SUB_R6
07     ST R7, SUB_R7;Return address
08
09     ;***Code***
10
11     ;Register Restoring
12     LD R0, SUB_R0
13     LD R1, SUB_R1
14     ...
15     LD R6, SUB_R6
16     LD R7, SUB_R7      ;Return address
17     RET
```

1 Review

2 Subroutines

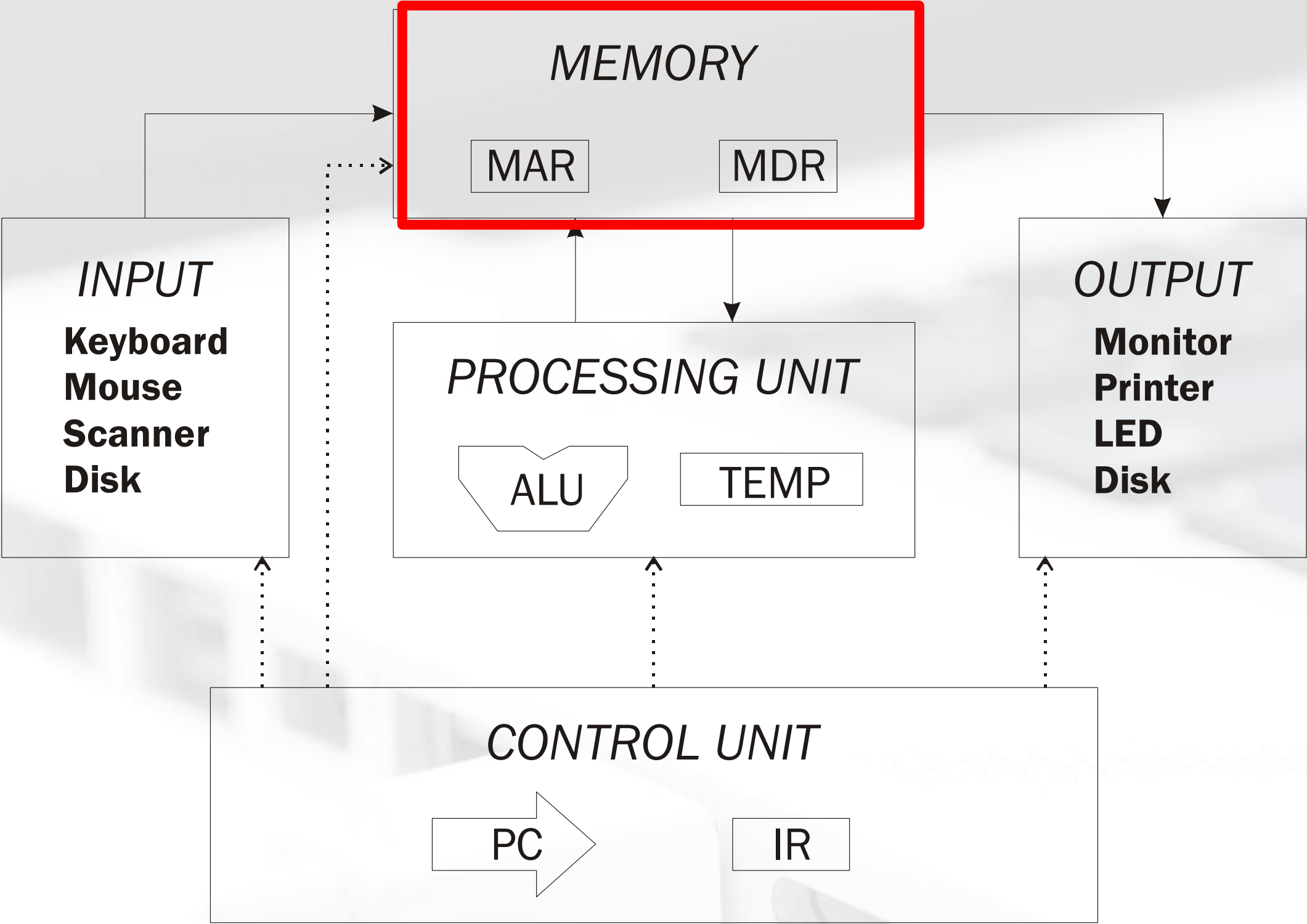
3 Control Instructions for Subroutines

4 **Memory Model for Program Execution**

5 The Stack

6 Implementing Functions in C

Review: Memory in Von Neumann Model



Review: Using Memory

■ Memory

- Just a big “array”
- “Indexed” by address
- Accessed with loads and stores instructions

■ LD/LDR/LDI

- Read a word out of memory
- Use different addressing mode

■ ST/STR/STI

- Place a word in memory
- Use different addressing mode

Memory

Address	Value
x0000	x00A0
x0001	x5007
x0002	x0201
x0003	x0203
x0004	x3002
...	
xFFFC	x5007
xFFFD	x0201
xFFFE	x0203
xFFFF	x3002

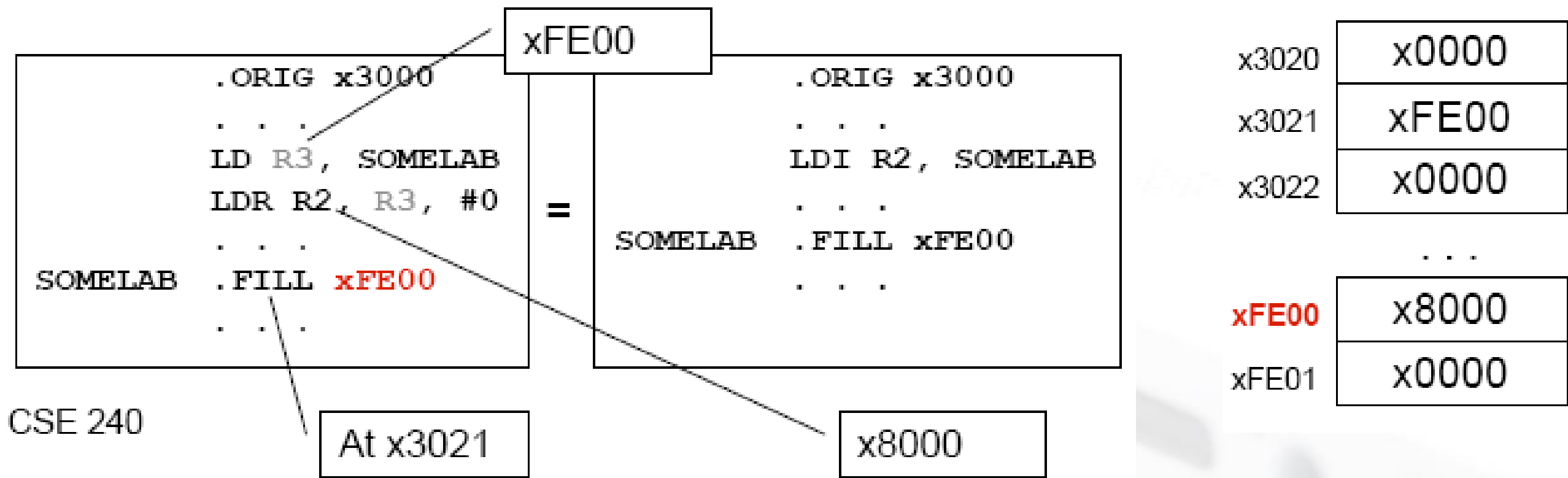
Review: Using Memory

■ Problem

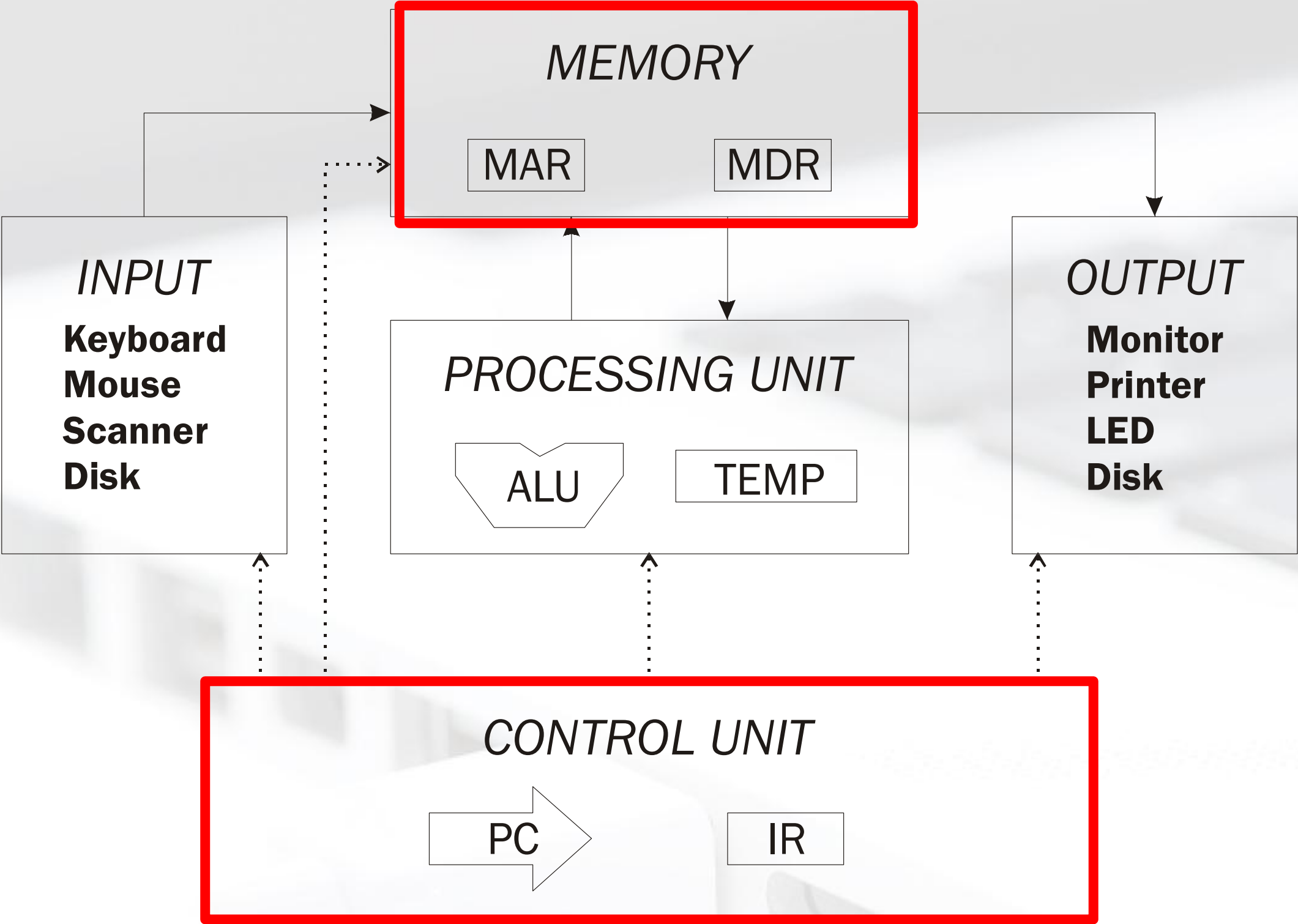
- What if the memory you want to access is far away?
- LD/ST won't work (PC-relative)
- LDR/STR won't work alone (need to get address in register)

■ Solution: LDI/STI

- Place *address* of far away value nearby
- Load address, then load/store from that



Memory Model for Function Calls



Problem

■ How do we allocate memory during the execution of a program written in C?

- Programs need memory for **code and data** such as instructions, global and local variables, etc.
- Modern programming practices encourage many **(reusable) functions**, callable from anywhere.
- Some memory **can be statically allocated**, since the size and type is known at compile time.
- Some memory **must be allocated dynamically**, size and type is unknown at compile time.

Motivation

■ Why is memory allocation important? Why not just use **a memory manager**?

- Allocation affects the performance and memory usage of every C, C++, Java program.
- Current systems do not have **enough registers** to store everything that is required.
- Memory management is too **slow and cumbersome** to solve the problem.
- Static allocation of memory resources is too **inflexible and inefficient**, as we will see.

■ What do we care about?

- Fast program execution
- Efficient memory usage
- Avoid memory fragmentation
- Maintain data locality
- Allow recursive calls
- Support parallel execution
- Minimize resource allocation
- Memory should never be allocated for functions that are not executed.

Scope: Local vs. Global

- A variable's declaration assists the compiler in managing the storage of that variable.
- In C, a variable's declaration conveys three pieces of information to the compiler:
 - *the variable's identifier and its type*
 - The first two of these, identifier and type, the C compiler gets explicitly from the variable's declaration.
 - *the variable's scope*——
 - The third piece, scope, the compiler infers from the position of the declaration within the code.
The scope of a variable is the region of the program in which the variable is “alive” and accessible.
 - The good news is that in C, there are only two basic types of scope for a variable. Either the variable is *global* to the entire program, or it is *local, or private*, to a particular block of code.

A C program that demonstrates nested scope.

```
1 #include <stdio.h>
2
3 int globalVar = 2; // This variable is a global variable
4
5 int main(void)
6 {
7     int localVar = 3; // This variable is local to main
8
9     printf("globalVar = %d, localVar = %d\n", globalVar, localVar);
10
11     // Creating a new sub-block within main
12     {
13         int localVar = 4; // This local to the sub-block within main
14
15         printf("globalVar = %d, localVar = %d\n", globalVar, localVar);
16     }
17
18     printf("globalVar = %d, localVar = %d\n", globalVar, localVar);
19 }
```

C allows this: as long as the different variables sharing the same name are declared in separate blocks.

If we compile and execute this code, the output generated looks as follows:

```
globalVar = 2, localVar = 3
globalVar = 2, localVar = 4
globalVar = 2, localVar = 3
```

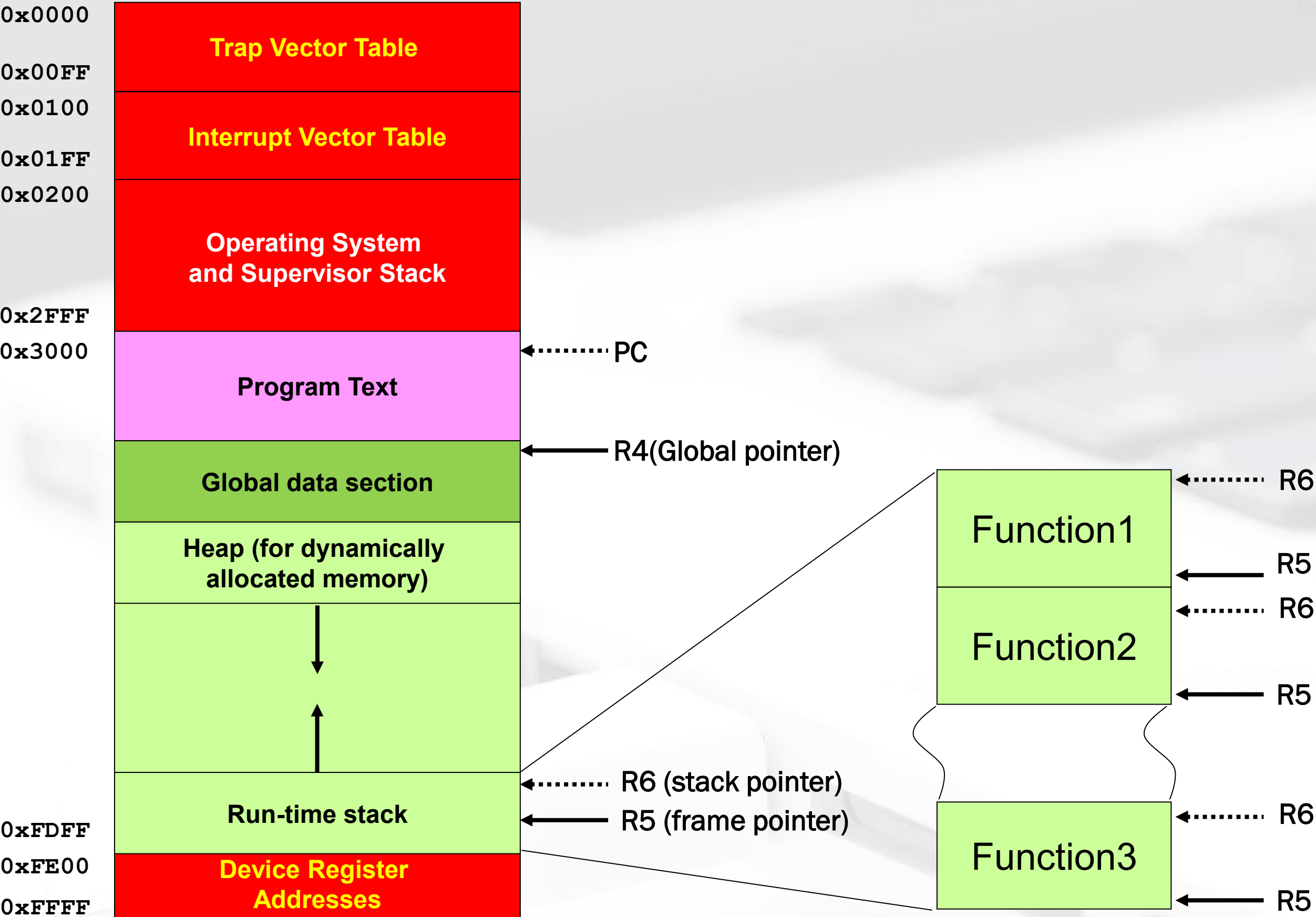
Initialization of Variables

```
double width;  
  
double pType = 9.44;  
double mass = 6.34E2;  
double verySmallAmount = 9.1094E-31;  
double veryLargeAmount = 7.334553E102;  
  
int average = 12;  
int windChillIndex = -21;  
  
int unknownValue;  
int mysteryAmount;  
  
bool flag = false;  
  
char car = 'A';    // single quotes specify a single ASCII character  
char number = '4'; // single quotes specify a single ASCII character
```

What initial value will a variable have if it has no initializer? In C, by default,

- **Local variables** start with an undefined value. That is, local variables have garbage values in them, unless we explicitly initialize them in our code. It is standard coding practice to explicitly initialize local variables within their declarations.
- **Global variables**, in contrast, are initialized to 0.

Memory Model in the LC-3



Allocating Space for Variables

■ There are two regions of memory in which declared variables in C are allocated storage:

- *the global data section*: Variables that are global are allocated storage in the global data section.
- *the run-time stack*: Local variables are allocated storage on the run-time stack.

A C program that performs a simple network rate calculation



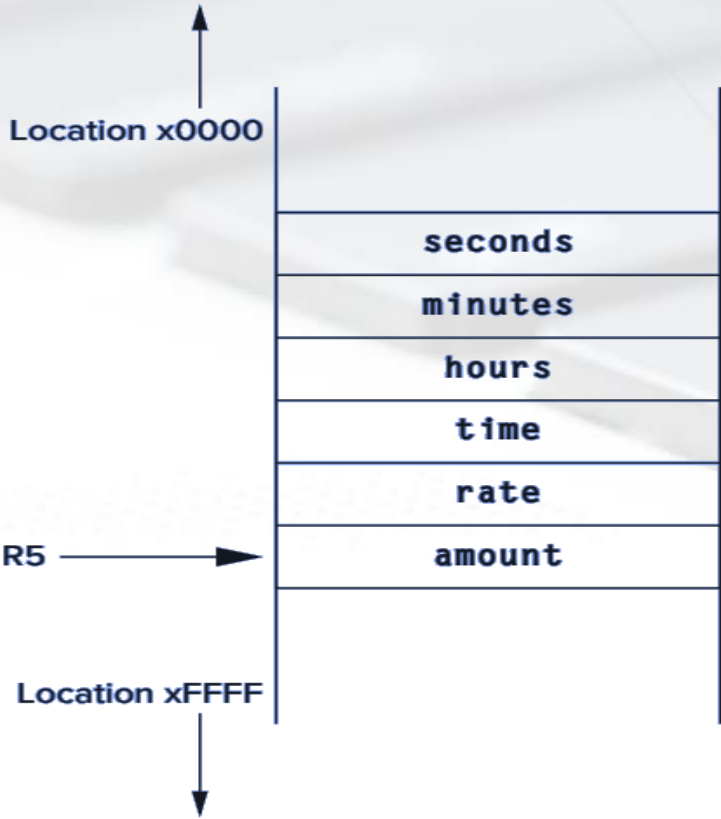
```
1 #include <stdio.h>
2 int main(void)
3 {
4     int amount;           // The number of bytes to be transferred
5     int rate;             // The average network transfer rate
6     int time;             // The time, in seconds, for the transfer
7     int hours;            // The number of hours for the transfer
8     int minutes;          // The number of mins for the transfer
9     int seconds;          // The number of secs for the transfer
10
11 // Get input: number of bytes and network transfer rate
12 printf("How many bytes of data to be transferred? ");
13 scanf("%d", &amount);
14 printf("What is the transfer rate (in bytes/sec)? ");
15 scanf("%d", &rate);
16
17 // Calculate total time in seconds
18 time = amount / rate;
19
20 // Convert time into hours, minutes, seconds
21 hours = time / 3600;      // 3600 seconds in an hour
22 minutes = (time % 3600) / 60; // 60 seconds in a minute
23 seconds = ((time % 3600) % 60); // remainder is seconds
24
25 // Output results
26 printf("Time : %dh %dm %ds\n", hours, minutes, seconds);
27 }
```

Identifier	Type	Location (as an offset)	Scope	Other info...
amount	int	0	main	...
hours	int	-3	main	...
minutes	int	-4	main	...
rate	int	-1	main	...
seconds	int	-5	main	...
time	int	-2	main	...

The compiler's symbol table when it compiles the code

The stack frame from function main of the code

- This function has five local variables.
- R5 is the frame pointer and points to the first local variable.



1

Review

2

Subroutines

3

Control Instructions for Subroutines

4

Memory Model for Program Execution

5

The Stack

6

Implementing Functions in C

Stack: An Abstract Data Type

- An important abstraction that you will encounter in many applications.
- The fundamental model for execution of C, Java, Fortran, and many other languages.
- We will describe two uses of the stack:
 - **Evaluating arithmetic expressions**
 - Store intermediate results on stack instead of in registers
 - **Function calls**
 - Store parameters, return values, return address, dynamic link
 - Interrupt-Driven I/O
 - Store processor state for currently executing program

Stack Data Structure

■ A LIFO (last-in first-out) storage structure

- The **first** thing you put in is the **last** thing you take out
- The **last** thing you put in is the **first** thing you take out
- This means of access is what defines a stack, not the specific implementation.

■ Two main operations

- **PUSH**: add an item to the stack
- **POP**: remove an item from the stack

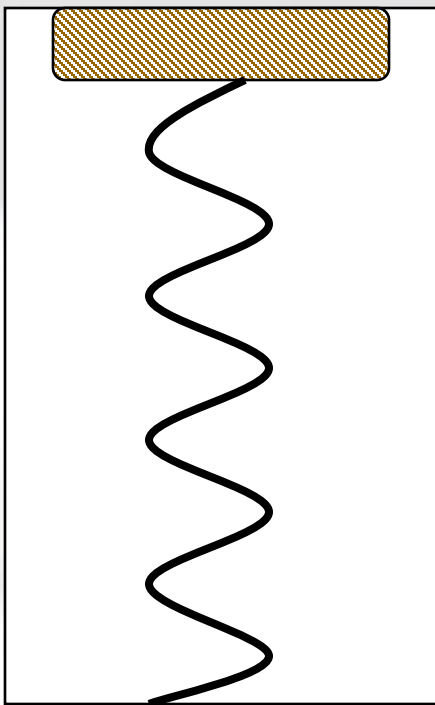
■ Error conditions:

- Underflow (try to pop from empty stack)
- Overflow (try to push onto full stack)

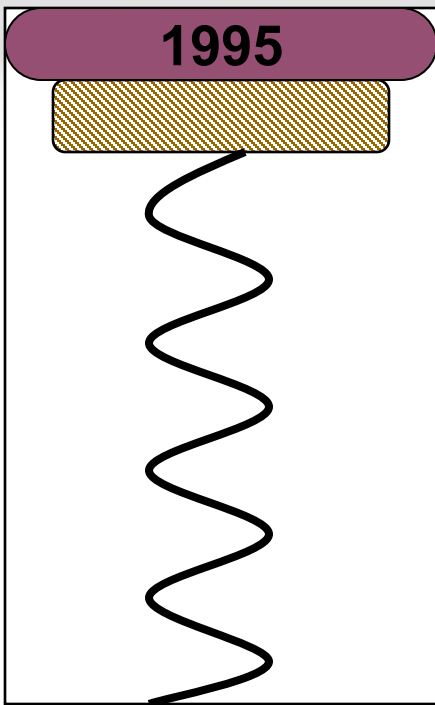
■ A register (eg. R6) holds address of top of stack (TOS)

A Physical Stack

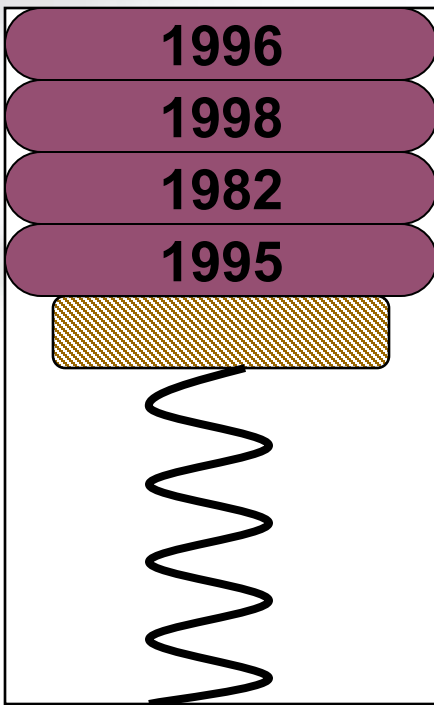
■ Coin holder



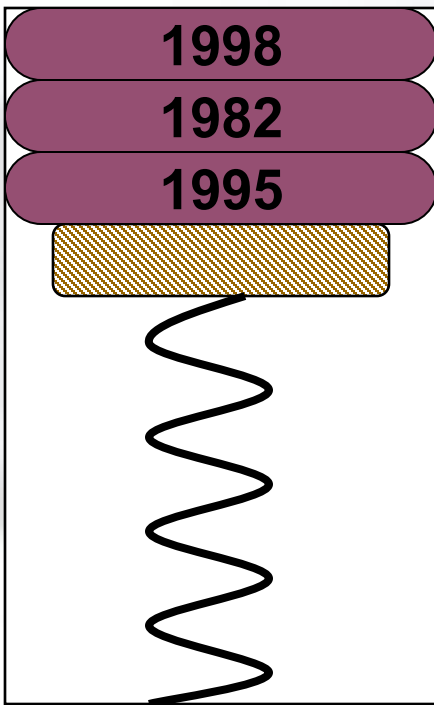
Initial State



After
One Push



After Three
More Pushes

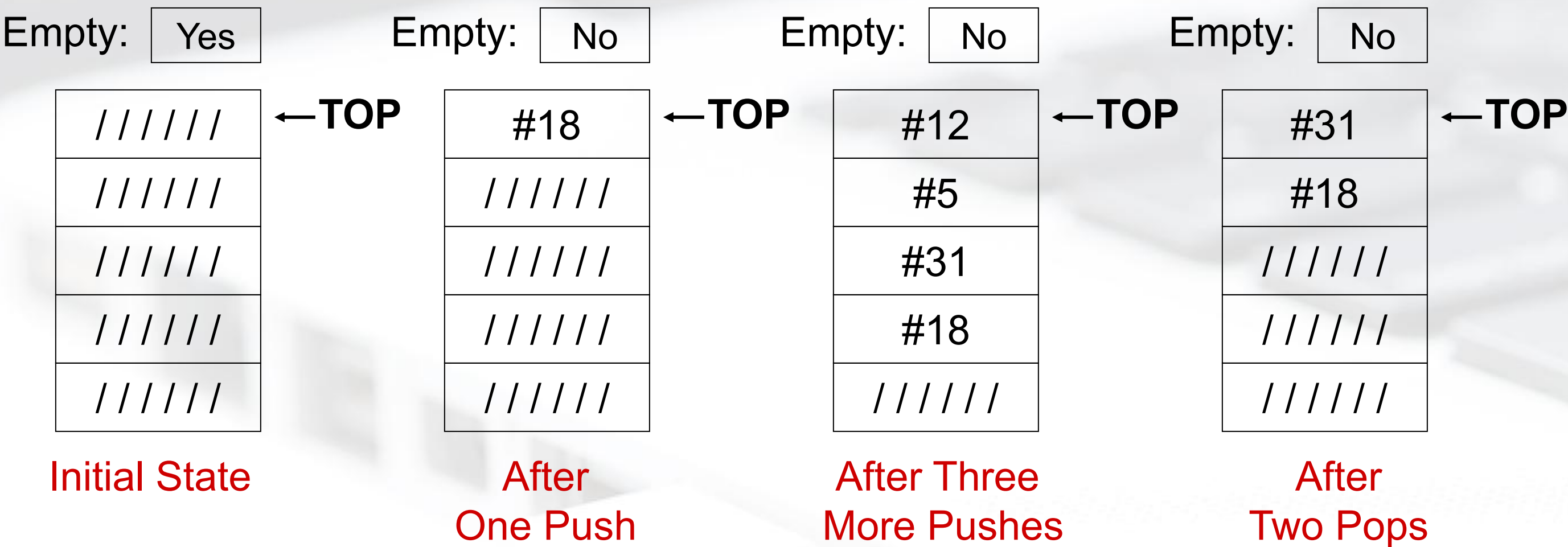


After
One Pop

Last quarter in is the first quarter out (LIFO)

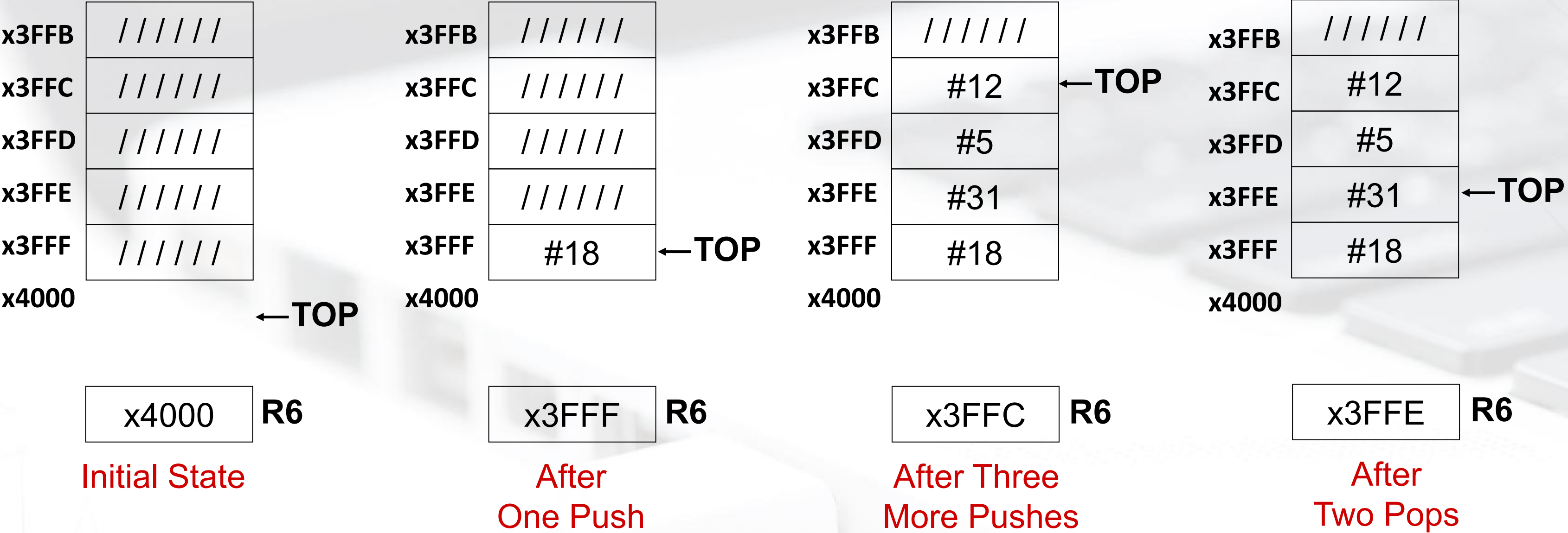
A Hardware Stack Implementation

■ Data items move between registers



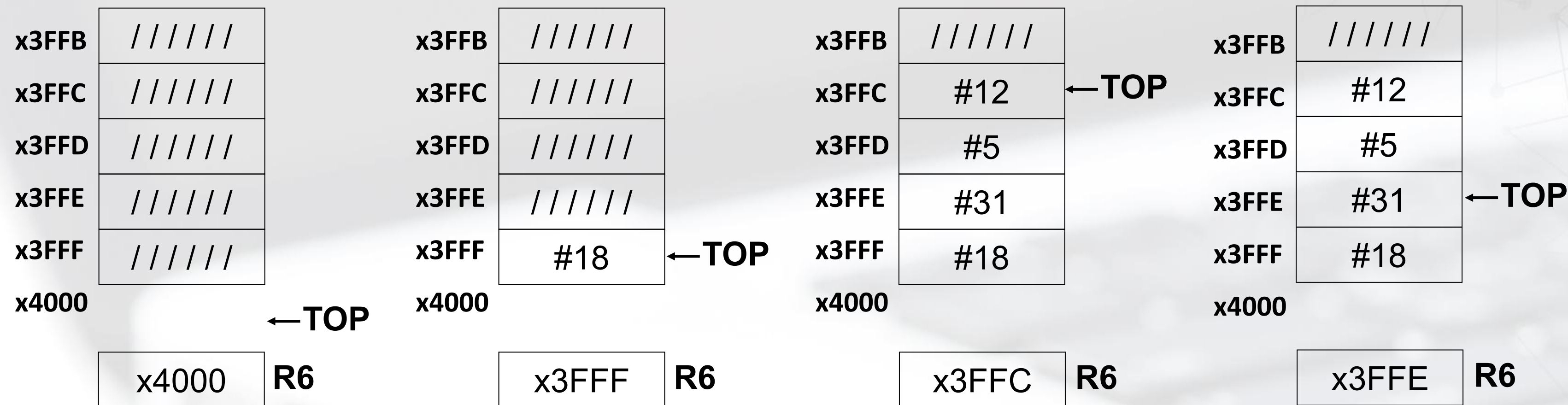
A Software Stack Implementation

■ Data items don't move in memory, just our idea about where TOP of the stack is



By convention, R6 holds the Top of Stack (TOS) Pointer (SP)

Basic Push and Pop Code



```

PUSH
    ADD R6, R6, #-1           ; increment stack ptr
    STR R0, R6, #0           ; store data(R0) to TOS

POP
    LDR R0, R6, #0           ; load data(R0) from TOS
    ADD R6, R6, #1           ; decrement stack ptr
  
```

■ **Note:** Stacks can grow in either direction (toward higher address or toward lower addresses)

Pop with Underflow Detection

■ If we try to pop too many items off the stack, an **underflow** condition occurs.

- Check for underflow by checking TOS before removing data.
- Return status code in R5 (0 for success, 1 for underflow)

```
POP    LD    R1, EMPTY
        ADD  R2, R6, R1          ; Compare stack pointer
        BRz  UNDER             ; with x3FFF
        LDR  R0, R6, #0         ; The actual 'pop'
        ADD  R6, R6, #1         ; Adjust stack pointer
        AND  R5, R5, #0         ; Success: return R5 = 0
        RET
UNDER  AND  R5, R5, #0          ; Underflow: return R5 = 1
        ADD  R5, R5, #1
        RET
EMPTY  .FILL xC000             ; EMPTY = -x4000
```

Push with Overflow Detection

■ If we try to push too many items onto the stack, an **overflow** condition occurs.

- Check for underflow by checking TOS before adding data.
- Return status code in R5 (0 for success, 1 for overflow)

```
PUSH  LD    R1, FULL
        ADD   R2, R6, R1          ; Compare stack pointer
        BRz   OVER               ; with x4004
        ADD  R6, R6, #-1         ; Adjust stack pointer
        STR  R0, R6, #0          ; The actual 'push'
        AND  R5, R5, #0         ; Success: return R5 = 0
        RET
OVER  AND  R5, R5, #0
        ADD  R5, R5, #1         ; Overflow: return R5 = 1
        RET
FULL  .FILL xC005             ; FULL = -x3FFB
```

The final code for PUSH & POP in LC-3 - 1

```

POP      ST R2, Save2           ; save, needed by POP
           ST R1, Save1           ; save, needed by POP
           LD R1, EMPTY           ; EMPTY contains -x3FFF
           ADD R2, R6, R1         ; Compare stack pointer with x3FFF
           BRz Fail_exit          ; Branch if stack empty
           LDR R0, R6, #0         ; The actual 'pop'
           ADD R6, R6, #1         ; Adjust stack pointer
           BRnzp Success_exit

EMPTY    .FILL xC000           ; EMPTY = -x4000

PUSH     ST R2, Save2           ; save, needed by PUSH
           ST R1, Save1           ; save, needed by PUSH
           LD R1, FULL            ;
           ADD R2, R6, R1         ; Compare stack pointer
           BRz Fail_exit          ; with x4004
           ADD R6, R6, #-1        ; Adjust stack pointer
           STR R0, R6, #0         ; The actual 'push'
           BRnzp Success_exit

FULL     .FILL xC005           ; FULL = -x3FFB

```

The final code for PUSH & POP in LC-3 - 2

```
Save1      .FILL x0000
Save2      .FILL x0000
Success_exit LD R1, Save1 ;Restore reg values
            LD R2, Save2 ;
            AND R5, R5, #0 ; Success: return R5 = 0
            RET

;
Fail_exit  LD R1, Save1 ;Restore reg values
            LD R2, Save2
            AND R5, R5, #0
            ADD R5, R5, #1 ; Overflow: return R5 = 1
            RET
```